Deriving Classic Recommendation Models and Losses, Part 1

From Two Towers to FM, DCN, DLRM, and DIN

Bo Hou

October 30, 2026

This is a detailed reasoning notebook reconstructed from a day of discussion about recommendation models. The aim is not to memorize names. It is to understand what relationships a model is supposed to learn, why its equations express those relationships, and what happens to the data and gradients.

The dialogue passages below are edited reconstructions, not verbatim transcripts. They preserve the substantive questions, objections, and corrections. Ambiguous notation and incorrect intermediate explanations have been corrected. These are learning notes about established methods, not claims of original research.

Contents

1. A common framework: data, modeling, optimization, and validation
2. Start with the recommendation problem, not a model name
3. Two-tower retrieval: inputs, architecture, and serving boundaries
4. User history: from weighted pooling to self-attention
5. Time cutoffs, overlapping windows, and representation caches
6. From concat plus MLP to explicit interactions
7. FM: replacing a category-pair coefficient table with inner products
8. DeepFM and Wide & Deep: two different combinations
9. DCN v1: expand the cross layers yourself
10. DCNv2: from a shared scalar to coordinate-specific mixing
11. Low-rank factorization and LoRA: what is constrained?
12. DLRM: preserve interaction scores and combine them nonlinearly
13. DIN: let the candidate determine which history matters
14. Losses covered so far: BCE, in-batch CE, logQ, and multiple positives
15. Appendix: significance, AUROC/NDCG, and feature importance
16. Where Part 2 should resume

1. A common framework: data, modeling, optimization, and validation

By the end of the discussion, I felt that most AI/ML problems could be approached through three questions:

1. **What data are we constructing?** What is one example? What are the features and labels? What distribution produced them?
2. **What do we want to learn, and how do we represent it?** Which relationships matter? Why does this architecture make those relationships accessible?

3. **How do we solve the learning problem?** What does the loss encourage? Where do gradients flow? Is the optimization stable and computationally practical?

A fourth question closes the loop: **How do we know it is useful?** That means offline evaluation, statistical uncertainty, online experiments, and serving cost.

This framework makes FM, DeepFM, DCN, and DLRM look less like unrelated tricks. They often use similar inputs and click labels. Several can use the same binary cross-entropy loss. Their differences lie largely in how the architecture represents interactions.

An MLP can represent complicated nonlinear functions. That does not mean every useful relationship is equally easy to learn from a finite dataset with a finite model and optimizer budget. Supplying an explicit product or inner product is an inductive bias: it makes a particular relationship easier to express.

The right explanation is not, "An MLP cannot learn this." It is, "This architecture makes this kind of relationship explicit, with a particular parameter-sharing pattern and computational cost."

2. Start with the recommendation problem, not a model name

2.1 Diagnose the popularity problem first

The initial scenario was a video recommendation product dominated by popular videos. The goal was to improve personalization and discovery of less popular content while preventing regressions in established metrics such as CTR.

My first ideas were history-based boosting, video-to-video propagation, and a lightweight two-tower model. But the more important initial question was where the problem originated.

My correction: Before proposing solutions, find out whether popularity concentration comes from retrieval, ranking, or insufficient exploration. Understand the existing system first.

A useful funnel inspection compares:

- The popularity distribution of the eligible video inventory.
- The distribution among retrieved candidates.
- The distribution after ranking, among actual exposures.
- The distribution among clicks or high-quality watches.

If retrieval never supplies tail items, ranking cannot promote missing candidates. If retrieval supplies them but they disappear before exposure, ranking scores and constraints deserve attention.

These distributions provide diagnostic evidence, not a complete causal explanation. Eligibility, inventory, user preference, filtering, and business rules can all change the funnel. A drop in entropy at one stage does not prove that stage is the root cause.

2.2 Diversity is not the same as reduced popularity bias

An early correction was that diversity does not automatically measure success against popularity concentration.

A recommendation list can cover many genres while containing only blockbuster videos. It can also contain only tail items that are almost identical to one another.

A more direct evaluation fixes a popularity definition before the experiment, such as clicks or exposures in a historical reference window. Then compare:

- Head, middle, and tail exposure shares and click shares.

- Click counts, CTR, and watch quality within each popularity bucket.
- Tail-item coverage and the number of distinct tail items discovered.
- How a fixed exposure budget is redistributed across buckets.

Use the same pre-experiment definition and bucket boundaries in both arms. Otherwise, the experiment can change the popularity counts that define the buckets, moving the evaluation target itself.

I initially described a desirable curve as "moving left." That statement is meaningful only after defining the axes. If the horizontal axis is fixed item popularity, increasing toward the right, then more useful engagement in the left-hand buckets could indicate greater tail discovery. But a histogram of item counts, a histogram of clicks, and a normalized exposure distribution answer different questions.

2.3 Build successive baselines

Variant	Main idea	What to check
Control	Current product	Keep a realistic reference
Base 1	Boost from the user's own click history	Reconsumption versus discovery; already watched does not always mean recommend again
Base 2	Propagate user-to-video signal through video-to-video links	Does propagation expand coverage or reinforce head items?
Base 3	Learn user and video vectors for ANN retrieval	Training distribution, loss, latency, and index freshness

Suppose the existing ranking formula is approximately

$$s(u, v) = w_1 p(\text{click} \mid u, v) + w_2 \text{popularity}(v).$$

An ablation or weight adjustment can test whether the explicit popularity term overwhelms personalization. That is a relatively cheap diagnostic experiment.

However, reducing a popularity weight is not automatically an exploration algorithm. It redistributes traffic and can reveal useful evidence. A deliberate exploration policy additionally concerns uncertainty, randomized decisions, and feedback over time.

3. Two-tower retrieval: inputs, architecture, and serving boundaries

3.1 The baseline I wanted to build

Potential user inputs include user ID, an existing learned user embedding, age, location, group membership, and an encoded behavior history.

Potential video inputs include video ID, an embedding supplied by the content team, text embeddings, audio embeddings, categories, creation time, and statistics available at the prediction time.

Concatenate features within each side, then project them into a shared space:

$$q_u = f_\theta(\text{user features}), \qquad k_v = g_\phi(\text{video features}),$$

$$q_u, k_v \in \mathbb{R}^d, \qquad s(u, v) = q_u^\top k_v.$$

L2 normalization and temperature scaling can be added when appropriate.

The feature list is a set of possibilities, not a prescription. More fields are not automatically better. Availability, incremental information, privacy requirements, serving cost, and update frequency matter. A field such as a user's name should not be included merely because it exists.

3.2 "Isn't that basically CLIP?"

My observation: We have two embeddings, train their encoders, and map both into a shared space. A richer feature set just means more inputs and small MLPs. Why split this into two separate model versions?

At the architectural level, the connection is reasonable: both use separate encoders and contrastive matching. But CLIP names a particular image-text pretraining method. A recommendation model with user and video features is more clearly described as a two-tower or dual-encoder model.

There is no need to pretend that a two-tower model using existing embeddings and one using additional sparse/ dense features are fundamentally unrelated methods. An extensible baseline can support both.

3.3 The important serving boundary: independent encoding

I initially imagined an API accepting `UserInfo`, `VideoInfo`, and `UserVideoInteractionInfo`. The crucial question is what each object contains and what the model actually consumes as prediction features.

A standard ANN two-tower retriever requires:

- Video vectors can be computed before a specific user request and indexed.
- The user query vector does not depend on the particular candidate currently being scored.
- A shared similarity function can search the index efficiently.

If the user tower consumes an expensive interaction computed specifically for the current user-candidate pair, it breaks that independent-encoding contract.

But the word "interaction" alone is not enough to disqualify a feature:

- **Past interactions** can be encoded in the user history.
- **Video-level statistics**, known at prediction time, can be video features.
- **The current pair's click label**, sample weight, sampling probability, and session metadata can support training without becoming tower inputs.
- **Candidate-dependent pairwise features** generally fit more naturally in ranking.

The boundary depends on information source, time, and candidate dependence. DIN later makes this distinction especially concrete.

4. User history: from weighted pooling to self-attention

4.1 A first baseline: action weights and time decay

Represent the history available before time t as

$$H_u(t) = \{(o_i, a_i, T_i) : T_i < t\},$$

where o_i is the object acted on, a_i is an action such as click, watch, or like, and T_i is the event time.

Encode an event or its content and action as e_i. A weighted pooling baseline is

$$h_u = \frac{\sum_i s(a_i)\,\delta(t - T_i)\,e_i}{\sum_i s(a_i)\,\delta(t - T_i)}.$$

The action mapping s can give a higher weight to a high-quality watch than to a brief click; δ can reduce the influence of old events.

An important distinction: a **past action** is valid historical input. The action we are currently trying to predict is the label. An action can serve as supervision in one training construction and as a known historical feature in another.

4.2 The comedy-versus-thriller objection

Discussion prompt: If a user likes both comedy and thrillers, averaging the two groups may put the user vector between them. Could this lose the user's distinct interests?

My objection: If all other videos are farther away, both groups could still be retrieved. The middle point is not automatically wrong.

That objection is valid. "Averaging can create a bottleneck" is not the same as "averaging necessarily fails." The outcome depends on the geometry, vector dimension, candidate distribution, and retrieval rule.

A more precise question is: **Does the aggregation discard distinctions the downstream task needs?**

With fixed event embeddings, two distinct histories can have the same average. If all other inputs are also identical, an MLP receiving only that average cannot distinguish the histories. Increasing the downstream parameter count does not recover information that is absent from the input.

Jointly training the event encoder can reduce such collisions by changing the representation. It does not turn a fixed aggregation operation into a lossless encoding of arbitrary histories.

4.3 A detour through GMMs and global clusters

I proposed keeping separate comedy and thriller centers, feeding several centers to an MLP, or letting a user embedding gate shared global clusters.

The idea introduced several different modeling questions:

1. Are we clustering users, videos, or interaction representations?
2. Are cluster centers shared globally or fitted separately for each user?
3. How are a user's individual events assigned to different interests?
4. Does the final retriever use one vector or multiple vectors?

Shared global clusters can be useful prototypes or an interest dictionary. But if gating receives only an already compressed average, it cannot guarantee recovery of distinctions lost before the gating step.

Fitting a separate GMM per user is not necessary to build a multi-interest model. It can also be expensive or statistically unreliable for short, frequently changing histories.

For completeness, the covariance representations discussed were:

GMM covariance type	Stored shape	Interpretation
Full	$K \times d \times d$	One full covariance matrix per cluster
Tied	$d \times d$	One covariance shared by every cluster
Diagonal	$K \times d$	Separate per-coordinate variances, no off-diagonal covariances
Spherical	K	One isotropic variance per cluster

Those are legitimate choices, but the main problem at that point was still how to encode a user's history. The discussion needed to converge rather than introduce GMM machinery just to arrive at multiple interests.

4.4 Returning to what I meant by attention

My clarification: I was not assuming independent embeddings followed by a fixed average. The original embeddings can pass through self-attention first. The final valid position can already contain historical context and become the user representation.

This resolves an important mismatch. "Weighting independent embeddings" and "encoding a sequence before extracting a state" are different operations.

A simplified causal attention layer is

$$E = [e_1; \ldots; e_T], \qquad Q = EW_Q, \quad K = EW_K, \quad V = EW_V,$$

$$\text{Attention}(E) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

Here M includes causal and padding masks. Position or time information is also necessary: **self-attention does not automatically know the order of an unannotated set of inputs.**

After the encoder stack,

$$h_u(t) = \text{HistoryEncoder}(H_u(t))_{\text{last}}$$

can be used as a user-tower feature.

Two corrections keep this interpretation precise:

- The last hidden state summarizes history for prediction. It need not literally be an embedding of a generated next action. We do not have to invent an additional action to obtain the state.
- Attention already performs learned aggregation, so we need not mechanically repeat the same pooling. But this does not prove that every system can dispense with pooling, explicit temporal features, or multi-interest representations.

5. Time cutoffs, overlapping windows, and representation caches

5.1 Leakage is more than an event-time comparison

The basic rule is that every prediction input must have been available at the prediction time t.

This includes:

- Historical events occur before the cutoff.
- The user embedding was not computed from future behavior.
- Action weights and time-decay statistics do not use future information.
- A video's final cumulative click count is not backfilled into earlier examples.
- If the evaluation simulates strict forward prediction, feature-encoder training data and model versions must also satisfy the stated protocol.

One protocol may use information before t to predict a later action. Another may label the action at t and cut features off immediately before that action. What matters is an explicit, consistent protocol, not an argument about the symbols $< t$ versus $\leq t$ in isolation.

5.2 One hundred events, windows of thirty

For a history of 100 events, use events 1-30 to predict event 31, then events 2-31 to predict event 32, and continue through events 70-99 predicting event 100. This produces 70 training examples.

Each example can jointly train the history encoder and the two-tower model. Adjacent windows can be placed in different batches; optimization does not have to follow the event order.

My question: If every example's features are earlier than its label, isn't that sufficient?

It is a necessary within-example condition. Evaluation imposes an additional boundary.

Consider randomly splitting overlapping windows:

- Test: events 1-30 predict event 31.
- Train: events 2-31 predict event 32.

The training example contains the test target event as part of its history. This is not a faithful simulation of training only on the past and predicting the future.

For forward-time evaluation, first define global training, validation, and test cutoffs, then construct windows consistent with that protocol. **The training partition can still be shuffled into random batches.**

Whether newly observed events before a test prediction may be included in that test history depends on the intended online process. They should not be excluded merely because they occur in the evaluation period, nor included without checking that they would really be available.

5.3 Overlapping windows still repeat computation

Putting adjacent windows in separate batches changes scheduling, not the amount of repeated history encoding.

During joint training, encoder parameters change. A cached state produced under an earlier parameter version is generally not interchangeable with a fresh computation under the current version, especially if we need the correct backward graph.

Possible efficiency choices include sampling windows, limiting history length, or supervising multiple positions from one sequence forward pass. Each has to align with labels, negative sampling, padding, and masks.

5.4 Joint training does not require synchronous online execution

My initial assumption: If the encoder and tower train together, they must run together online; if they train separately, they must serve separately.

The correction is that **training dependencies and serving schedules are different things**.

The history encoder and user tower can train jointly. At serving time, an expensive history representation can be precomputed and cached. A request reads that representation and combines it with current context in a lighter user tower.

This creates operational requirements around freshness and compatibility: history-cache version, encoder version, tower version, feature version, and video-index version.

If the user representation includes today's observations while the indexed video vectors were generated yesterday, that is not automatically feature leakage. When both contain only information available at the actual request time, the issue is more likely staleness, drift, or representation compatibility.

5.5 New events and KV caching

A causal encoder can sometimes reuse previous positions' keys and values when a new event arrives. The new position computes its state while attending to the cached history.

However, it is inaccurate to say that "only the final $QK^\top$ changes." At every layer, the new position produces new hidden states and Q/K/V and executes the remaining layer transformations. A new model version generally invalidates caches built by the old one.

A strict sliding window adds another subtlety. Existing states may already contain information from an event that has now left the window. Dropping only that event's KV entries does not necessarily reproduce a full recomputation over the shortened history. The cache semantics must match the context definition.

6. From concat plus MLP to explicit interactions

6.1 "The MLP already has strong capacity"

My first response: Concatenate two embeddings, apply an MLP, and add a hidden layer if necessary. Why is explicit crossing needed?

The response should not deny MLP expressiveness. The real motivation is to expose multiplication-like relationships directly instead of relying on the network to discover them indirectly.

For two vectors $e_f, e_g \in \mathbb{R}^{16}$, their outer product is

$$e_f e_g^\top \in \mathbb{R}^{16\times 16}.$$

Each entry $(e_f)_a (e_g)_b$ is a product between a coordinate of one representation and a coordinate of the other.

Embedding coordinates do not generally have fixed human meanings such as "comedy" or "age." The product structure makes latent-coordinate relationships available to the model; their interpretation emerges from training.

6.2 "Compress the 256 products with a linear layer"

Flatten the 16-by-16 outer product and apply a 256-to-1 linear map. This gives

$$s_{fg} = \text{vec}(e_f e_g^\top)^\top \text{vec}(W) = e_f^\top W e_g,$$

where $W \in \mathbb{R}^{16\times 16}$.

This is a bilinear score. The same W can be reused for every field pair. Sharing means every application refers to the same parameter tensor, so gradients from the pairs accumulate into that tensor.

```
# E: [batch, num_fields, d]
# W: [d, d]
projected = E @ W
scores = projected @ E.transpose(-1, -2)
# scores[b, f, g] = E[b, f] @ W @ E[b, g]
# Keep f < g. If W is asymmetric, fix a field ordering.
```

W has only d^2 parameters. The number of pair scores can nevertheless grow quadratically with the field count. Parameter size and computation are separate questions.

7. FM: replacing a category-pair coefficient table with inner products

7.1 Fix the notation before going further

The largest source of confusion was reusing the same matrix name for a 16-by-16 embedding-space transformation and a 10,000-by-10,000 category-space coefficient table.

Symbol	Meaning	Running example
F	Number of fields	100
C_f	Categories in field f	100 per field
$M = \sum_f C_f$	Total concatenated one-hot dimension	10,000
d	Embedding dimension	16
$D = Fd$	Length of concatenated active embeddings	1,600
W	Bilinear transformation between embedding coordinates	16 by 16
C	Coefficients between raw categorical coordinates	10,000 by 10,000

These are different objects. A matrix did not suddenly grow by four orders of magnitude; the notation had switched spaces without making that explicit.

7.2 What are a field and a category?

A **field** is an attribute, such as location, language, or device. A **category** is one value within that field, such as New Jersey, English, or iPhone.

One hundred fields with one hundred categories each does not mean that every example activates ten thousand features. With one category selected per field, the concatenated one-hot vector has dimension 10,000 but only 100 active positions.

In implementation, keep category IDs and look up the active embeddings:

$$e_f = E_f[\text{category_id}_f].$$

There is no need to allocate a dense one-hot vector.

Hashing maps raw categories into a finite ID space before lookup. It is not the same as interval-based binning of a continuous feature, and it can introduce collisions.

7.3 Why does a direct category-pair table become large?

For two fields with 100 categories each, there are 10,000 possible category combinations.

For 100 fields,

$$\binom{100}{2} = 4,950$$

distinct field pairs exist. Learning an independent coefficient for every cross-field category pair therefore requires

$$4,950 \times 10,000 = 49,500,000$$

second-order coefficients.

A full 10,000-by-10,000 matrix has 100 million entries, but includes symmetric duplicates, diagonal entries, and within-field combinations. It would be misleading to call all 100 million independent useful interaction coefficients in the one-category-per-field setting.

My objection: The one-hot inputs are extremely sparse. Doesn't that solve the problem?

It reduces the work needed for one example. It does not automatically eliminate storage for an independent coefficient table or make rarely observed coefficients statistically easy to estimate.

7.4 FM's factorization

Start with a second-order predictor:

$$s = b + \sum_{i=1}^{M} w_i x_i + \sum_{i<j} C_{ij} x_i x_j.$$

FM replaces the pair coefficient by

$$C_{ij} = \langle v_i, v_j \rangle, \qquad v_i \in \mathbb{R}^d,$$

giving

$$s_{\text{FM}} = b + \sum_i w_i x_i + \sum_{i<j} \langle v_i, v_j \rangle x_i x_j.$$

Here i and j index **category coordinates in the concatenated input**, not fields. In a one-hot encoding, $x_i x_j$ selects pairs of active categories.

For single-valued categorical fields, the lookup-based expression we had been discussing is simpler:

$$s_{\text{interaction}} = \sum_{f<g} \langle e_f, e_g \rangle.$$

My clarification: I meant that each field chooses its embedding, we train those embeddings, and then take inner products. I was not asking to explicitly construct the raw one-hot matrix again.

Exactly. Lookup already performs the category selection represented by the indicators in the original formula.

The embedding table has $Md = 10,000 \times 16 = 160,000$ parameters, rather than one independently estimated coefficient for each of approximately 49.5 million category pairs.

7.5 "So we force W to be the identity?"

Relative to the earlier bilinear score,

$$e_f^\top W e_g,$$

an inner product uses

$$e_f^\top I e_g = e_f^\top e_g.$$

This is a useful architectural interpretation: omit the explicit W and let the embeddings learn relationships.

It is not a proof that every arbitrary W can be absorbed without loss into one shared embedding representation. General bilinear forms can express asymmetric or indefinite relationships; a shared dot-product parameterization has its own constraints.

FM's benefit is a structured, factorized parameterization with shared latent factors. It is not an unrestricted re-expression of every possible category interaction table.

When describing its parameter size as N-by-K, define the symbols: N here is the number of raw input coordinates M, and K is the latent vector dimension d. K does not mean the number of independent embedding tables.

7.6 Is there a learned weight after each inner product?

This question came up repeatedly and needs an explicit answer.

Standard FM:

$$\sum_{f<g} \langle e_f, e_g \rangle.$$

The pair scores are summed directly. There is no additional field-pair coefficient after each inner product. The embeddings are learned, and the first-order branch has its own learned coefficients.

The extension I proposed:

$$\sum_{f<g} a_{fg} \langle e_f, e_g \rangle.$$

The a_{fg} values are also learned. That is a legitimate weighted extension, but not the standard second-order FM equation.

There are two separate meanings of "learned interaction weight":

1. The inner product itself is the learned coefficient of a raw category cross.
2. An extra field-pair coefficient can weight that inner-product output.

Only the first is required in standard FM. Keeping these meanings separate prevents a great deal of confusion.

7.7 Why FM can sum interactions without enumerating all pairs

The identity

$$\sum_{i<j}\langle v_i, v_j\rangle x_i x_j = \frac{1}{2}\sum_{k=1}^{d}\left[\left(\sum_i v_{ik}x_i\right)^2 - \sum_i v_{ik}^2 x_i^2\right]$$

lets us calculate the second-order sum in $O(m_{\text{active}}d)$ time, where m_{active} is the number of nonzero coordinates.

If we need every pair score separately, or introduce arbitrary independent a_{fg}, we cannot automatically reuse this same simplification.

7.8 How lookup appears in a neural network

```python
import torch
from torch import nn

class LookupAndFM(nn.Module):
    def __init__(self, num_fields=100, categories_per_field=100, d=16):
        super().__init__()
        self.tables = nn.ModuleList([
            nn.Embedding(categories_per_field, d)
            for _ in range(num_fields)
        ])

    def forward(self, category_ids):
        # category_ids: [B, F]; one selected category per field
        E = torch.stack([
            table(category_ids[:, f])
            for f, table in enumerate(self.tables)
        ], dim=1)                       # [B, F, d]
        sum_square = E.sum(dim=1).square()
        square_sum = E.square().sum(dim=1)
        score = 0.5 * (sum_square - square_sum).sum(dim=-1)
        return score                   # [B], interaction branch only
```

Lookup is trainable. It is equivalent to multiplying a one-hot input by an embedding matrix, without materializing the one-hot input. Autograd accumulates gradients into the table rows selected by the batch.

This snippet illustrates the tensor shapes and interaction calculation. It is not a complete training system and was not executed as an end-to-end implementation for this article.

8. DeepFM and Wide & Deep: two different combinations

8.1 DeepFM does not enumerate every triple

My question: With 100 fields, FM has 4,950 inner products. Does DeepFM need $\binom{100}{3}$ three-field combinations?

No. Its main construction is a factorization-machine branch and a deep branch with shared embeddings.

For an illustrative deep branch,

$$z = \text{concat}(e_1, \dots, e_F, \text{dense inputs}),$$

$$h_1 = \sigma(W_1 z + b_1), \qquad h_2 = \sigma(W_2 h_1 + b_2),$$

$$s_{\text{deep}} = w_o^\top h_2 + b_o,$$

$$s = s_{\text{FM}} + s_{\text{deep}}, \qquad p = \text{sigmoid}(s).$$

Two hidden layers are an example, not a defining requirement. When counting layers, specify whether the final linear output layer is included.

The FM branch sums second-order interactions. The deep branch receives the original embedding concatenation and learns nonlinear combinations. It does **not** require feeding the 4,950 FM pair scores into an MLP first.

An MLP can represent relationships involving three, four, or many inputs. That does not imply it will learn every such interaction adequately. Nor does it give the explicit maximum polynomial degree that a cross network provides.

8.2 Is the probability an intermediate feature or the final prediction?

In a classic CTR formulation, the FM and deep branches produce scalar logit contributions. Their sum is passed through a sigmoid to obtain a final click probability.

It is possible to change the deep branch into an embedding encoder and connect it to a larger network. Other features can also be incorporated. Those are sensible extensions, but they should be distinguished from the standard architecture being explained.

8.3 Wide & Deep: define cross features, learn their coefficients

My suggestion: If certain combinations matter, why not add a rule-related term to the model score?

The wide branch can be written as

$$s_{\text{wide}} = b + \sum_j a_j \phi_j(x),$$

where $\phi_j(x)$ may be a raw feature or an explicitly constructed cross indicator. For example, a particular region and topic category being present together may activate one indicator.

$$s = s_{\text{wide}} + s_{\text{deep}}, \qquad p = \text{sigmoid}(s).$$

The human-selected part is which cross transformations are made available. Their coefficients a_j can be learned jointly through the prediction loss. This differs from hard-coding a post-model business-rule boost.

FM parameterizes interactions through shared latent factors. A wide branch can assign individual coefficients to selected concrete combinations. They differ in parameter sharing and generalization behavior.

9. DCN v1: expand the cross layers yourself

9.1 What exactly is x0?

With 100 fields and 16-dimensional embeddings,

$$x_0 = \text{concat}(e_1, \dots, e_{100}) \in \mathbb{R}^{1600}.$$

This is the concatenated embedding representation, **not** the raw one-hot input. We do not need to compress it back into 16 dimensions first.

The vector x_l is the state at cross layer l, still D-dimensional. The subscript l denotes a layer, not a field.

9.2 "Is x0 to x1 the core? Let me derive it"

One way to start is an outer product:

$$x_0 x_l^\top \in \mathbb{R}^{D \times D}.$$

Multiply it by a learned $D \times 1$ vector w_l, returning a D-vector, and add a residual:

$$x_{l+1} = x_l + x_0(x_l^\top w_l) + b_l.$$

Associativity matters computationally. We can first calculate the scalar $x_l^\top w_l$, then multiply it by x_0. There is no need to materialize the D-by-D outer product. Each cross layer has O(D) parameters and arithmetic for this operation.

9.3 "But that just rescales x0"

Ignore biases temporarily and define

$$a_l = x_0^\top w_l.$$

The first layer is

$$x_1 = x_0 + x_0(x_0^\top w_0) = x_0(1 + a_0).$$

For the second layer,

$$\begin{aligned} x_2 &= x_1 + x_0(x_1^\top w_1) \\ &= x_0(1 + a_0) + x_0(1 + a_0)a_1 \\ &= x_0(1 + a_0)(1 + a_1) \\ &= x_0 + x_0a_0 + x_0a_1 + x_0a_0a_1. \end{aligned}$$

My objection: Everything is still x0 plus scaled copies of x0. Is this just a measure of how much original information is retained? Where are the second- and third-order crosses?

The answer becomes clear only when we expand a coordinate:

$$(x_0a_0)_i = \sum_j (w_0)_j(x_0)_i(x_0)_j.$$

That term is second order. And

$$(x_0a_0a_1)_i = \sum_{j,k} (w_0)_j(w_1)_k(x_0)_i(x_0)_j(x_0)_k$$

is third order.

Both of the following statements are therefore true:

1. Without biases, each intermediate state is collinear with x0, a strong expressive restriction.
2. The scale is itself a function of the input. It is not a constant, so the resulting function contains higher-order input products.

The moment it clicked: The point is to construct terms like $x_ix_jx_k$. Writing them as an input-dependent scale hides those terms until we expand the expression.

9.4 Depth, residuals, and interaction degree

L cross layers have a maximum explicit polynomial degree of L+1. Two cross layers therefore reach cubic terms in the cross branch.

This is a statement about coordinate-product degree, not necessarily about three distinct fields. Coordinates from the same field or repeated coordinates may occur.

The residual keeps lower-order paths. It does not imply that arbitrarily deep cross stacks are always better. Estimation difficulty, stability, regularization, and sample size remain relevant.

If a nonlinear deep network follows the cross layers, the complete model should not be described as "limited to third-order interactions" merely because it contains two cross layers.

10. DCNv2: from a shared scalar to coordinate-specific mixing

10.1 The natural next question

If v1 computes one scalar that modulates every coordinate, can different output coordinates receive different mixtures of the current state?

Replace the vector parameterization by a full matrix:

$$g_l = W_l x_l, \qquad W_l \in \mathbb{R}^{D \times D},$$

$$x_{l+1} = x_l + x_0 \odot (W_l x_l + b_l).$$

This uses the common DCNv2 convention with the bias inside the elementwise multiplication. Do not confuse it with the externally added bias in the earlier v1 equation.

My shape check: xl goes through a 1,600-by-1,600 matrix, producing 1,600 values, which are multiplied elementwise by x0?

Yes. "Modulation values" is more precise than probabilities: they need not be positive or sum to one.

In coordinates,

$$(x_{l+1})_i = (x_l)_i + (x_0)_i \left(\sum_j W_{l,ij} (x_l)_j + b_{l,i} \right).$$

V1 uses the same scalar aggregation for every coordinate. V2 lets each output coordinate learn a different mixture.

10.2 Does more data imply a fixed ranking of architectures?

My question: As the dataset grows, should we expect DCNv2 > DCN > DeepFM > FM?

There is no universal ordering. Results depend on features, sparsity, interaction structure, parameter count, tuning, regularization, optimizer settings, and implementation.

A paper showing an improvement under a particular setup provides evidence for that setup. It does not establish a sample-size threshold beyond which the same ranking must hold.

For industrial comparisons, check equalized compute budgets, meaningful baselines, fair tuning, effect sizes, and repeated experiments rather than treating publication order as a performance hierarchy.

11. Low-rank factorization and LoRA: what is constrained?

11.1 Factorizing a DCNv2 matrix

A D-by-D matrix requires D^2 parameters per layer. Parameterize

$$W_l = U_l V_l^\top, \qquad U_l, V_l \in \mathbb{R}^{D \times r}.$$

Then

$$W_l x_l = U_l (V_l^\top x_l).$$

The parameter count becomes 2Dr. The entire matrix W is constrained to rank at most r.

11.2 "Is LoRA the same idea? Wait, it is delta W"

That correction is the essential distinction:

Setting	Parameterization	Low-rank restriction
Factorized weight training	$W = BA$	The entire W
LoRA	$W = W_0 + \frac{\alpha}{r}BA$	The update ΔW

Here $B \in \mathbb{R}^{m\times r}$, $A \in \mathbb{R}^{r\times n}$, and A and B are directly trained. In ordinary LoRA, the existing W_0 is frozen.

BA is a parameterization of the update. Training does not require first computing a full delta matrix and then taking its SVD.

From-scratch training can also parameterize W itself as BA. But this limits the full layer's capacity. Rank-8 or rank-16 LoRA experience should not be transferred uncritically to native low-rank pretraining.

11.3 A side question about LLM pretraining experiments

The discussion briefly examined Spectron, which trains non-embedding matrices in a native factorized form. Its 94M-model rank ablation reports similar perplexities for input-dimension ratios of 0.25 and 0.4, with substantial deterioration at 0.125. See reference [9], Appendix B.2.

That is a concrete experimental starting point, not a universal industrial rule. The optimizer, model scale, and training budget all matter.

Projecting gradients into a low-rank space or accumulating multiple low-rank updates into high-rank weights is not the same as keeping the entire weight matrix permanently low rank.

This is a useful connection, but it is a side branch. The main discussion returns to recommendation architectures.

12. DLRM: preserve interaction scores and combine them nonlinearly

12.1 Start with sparse and dense inputs

My response to the input question: Age can be bucketed; user ID can be hashed; then both can use embedding lookup.

That is a valid representation choice. Another choice is to keep numerical inputs continuous, normalize or transform them as appropriate, and pass them through a bottom MLP:

$$h_{\text{dense}} = \text{MLP}_{\text{bottom}}(x_{\text{dense}}) \in \mathbb{R}^d.$$

Normalization does not necessarily mean forcing every coordinate into [0,1]. Standardization and logarithmic transforms can be appropriate depending on the distribution and meaning.

Sparse fields produce $e_1, \ldots, e_F \in \mathbb{R}^d$. Giving the dense branch the same output dimension lets it participate in inner-product interactions with those embeddings.

12.2 "Then we can attach FM, DeepFM, or DCN"

Yes, as architecture choices. But the classic DLRM construction makes a particular choice:

$$\mathcal{E} = [h_{\text{dense}}, e_1, \ldots, e_F],$$

$$p_{fg} = \langle \mathcal{E}_f, \mathcal{E}_g \rangle, \qquad f < g,$$

$$z = \text{concat}\left(h_{\text{dense}}, [p_{fg}]_{f<g}\right),$$

$$s = \text{MLP}_{\text{top}}(z), \qquad p = \text{sigmoid}(s).$$

With 100 sparse fields plus one dense representation, there are 101 interacting vectors and $\binom{101}{2} = 5,050$ pair scores. That is not the same as 4,950 pairs from only 100 vectors.

12.3 "Isn't DeepFM doing this too?"

The difference is what enters the MLP:

Architecture	MLP input	Treatment of pair scores
FM	No deep branch required	Direct sum
Weighted FM extension	No MLP required	Learned linear combination
DeepFM	Original embedding concatenation	Separate summed FM branch contributes to final logit
DLRM	Individual inner products plus dense representation	Top MLP learns nonlinear combinations

My conclusion: So the weighted version I described combines inner-product terms linearly, whereas DLRM combines them through an MLP.

Correct, provided that the extra learned pair weights are identified as an extension rather than silently attributed to standard FM.

12.4 Why expose these scores first?

The top MLP receives explicit pairwise matching signals rather than needing to construct the same multiplication relationships from raw embeddings.

It can learn conditions such as "boost only when both matching signals are high," rather than adding every score independently.

There is a tradeoff: one scalar per vector pair loses information. This does not guarantee superiority over concatenating the original vectors.

Nor is the representation always smaller. Concatenated embeddings have size roughly Fd; all pair scores have size O(F squared). When the field count is large and d is small, explicit pair scores can be longer. Substitute the actual dimensions before claiming compactness.

13. DIN: let the candidate determine which history matters

13.1 "Is this user behavior modeling?"

Yes. We still start with an action list whose events have embeddings e_i.

What changes is the question: **Which parts of the same history matter for this candidate?**

- For a comedy candidate, comedy-related history may matter more.
- For a thriller candidate, thriller-related history may matter more.

The desired representation becomes

$$h_u(v) = \text{Aggregate}(e_1, \dots, e_T \mid v).$$

It is candidate-dependent, unlike a single query vector built independently of every item being scored.

13.2 First answer: "Use an inner product"

My immediate relevance score was

$$s_i = e_i^\top v.$$

Then use the score to weight the historical embedding:

$$h_u(v) = \sum_i s_i e_i.$$

That is a sensible starting model. Raw dot products can be negative and change scale, however. Whether to apply an activation, normalization, or another stabilization step must be a deliberate choice. They are not automatically probability weights.

13.3 Next answer: "Concatenate ei and v, then use an MLP"

To make the relevance function more flexible,

$$s_i = \text{MLP}_{\text{activation}}([e_i, v]), \qquad h_u(v) = \sum_i s_i e_i.$$

The same relevance network is shared across history positions. The final task teaches it which historical behavior is useful for which candidate.

This captures DIN's core local-activation idea. The original paper includes explicit interaction inputs, the Dice activation, and mini-batch-aware regularization. The equation above is a teaching abstraction, not a full layer-by-layer reproduction.

DIN's local activation does not require softmax normalization. Do not assume that every mechanism called attention must produce weights summing to one.

13.4 Final answer: "Combine it with the candidate embedding"

Once the candidate-specific user representation exists,

$$s = \mathrm{MLP}_{\mathrm{rank}}([h_u(v), v, \text{other features}]),$$

$$p(\text{click}) = \mathrm{sigmoid}(s).$$

A click BCE loss can jointly train the rank MLP, activation network, and history/candidate embeddings through backpropagation.

13.5 How it differs from the self-attention user tower

Aspect	Candidate-independent history encoder	DIN-style aggregation
Main relation	Relationships within history	Relevance of history to a candidate
Output	A candidate-independent user state	A different interest representation per candidate
Standard ANN dual encoder	Directly compatible	Not directly a single independent query vector
Typical role	Retrieval or ranking user features	Ranking

They can be combined: encode the history with a sequence model and then aggregate with respect to a candidate. That is an extension, not a reason to call every attention-based user model DIN.

14. Losses covered so far: BCE, in-batch CE, logQ, and multiple positives

This section collects the objectives already discussed. The broader loss-family derivations belong in Part 2.

14.1 Pointwise click prediction: BCE

For an exposed example with label $y_i \in \{0, 1\}$, let the model output a logit s_i:

$$p_i = \mathrm{sigmoid}(s_i),$$

$$\ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i).$$

For nonnegative example weights ω_i, one possible normalization is

$$L = \frac{\sum_i \omega_i \ell_i}{\sum_i \omega_i}.$$

This expresses a per-example probability-prediction objective. FM, DeepFM, Wide & Deep, DCN, DLRM, and DIN can all use it. Architecture and objective are not in one-to-one correspondence.

Use a numerically stable logits-based implementation in practice. If negatives are sampled, calibration to the real exposure distribution requires attention to the sampling mechanism.

14.2 Two-tower training with in-batch negatives

Take B positive interactions (u_i, v_i), and calculate

$$S_{ij} = \frac{q_{u_i}^\top k_{v_j}}{\tau}.$$

The simplest one-positive-per-row objective is

$$\ell_i = -\log \frac{\exp(S_{ii})}{\sum_{j=1}^{B} \exp(S_{ij})}.$$

The B-by-B matrix has B diagonal positives and B(B-1) off-diagonal candidate negative pairings.

Those off-diagonal pairs are a training construction. They are not proof that the user dislikes every other batch item. They are sampled negatives or unobserved pairings, and false negatives are possible.

14.3 "The batch is random; don't distort the real data"

I repeatedly emphasized that random batching can naturally produce repeated users, repeated videos, and multiple positives. We should not alter genuine observations merely to make a toy formulation cleaner.

That concern is compatible with correctly defining the loss. If an identical item is positive in one position but treated as a negative in another, the objective can impose contradictory supervision. Item identity, positive sets, valid candidates, and duplicate handling need an explicit protocol.

For a known positive set P_i, one objective is

$$\ell_i = -\log \frac{\sum_{j \in P_i} \exp(S_{ij})}{\sum_{j \in A_i} \exp(S_{ij})},$$

where A_i is the valid candidate set for row i. This encourages probability mass on the positive set as a whole.

Another objective supervises each positive separately:

$$\ell_i = -\frac{1}{|P_i|} \sum_{p \in P_i} \log \frac{\exp(S_{ip})}{\sum_{j \in A_i} \exp(S_{ij})}.$$

These are not equivalent. The first can be satisfied mainly by one strong positive. The second more explicitly encourages every positive to receive probability. "Use a multi-positive loss" is incomplete without stating which behavior is intended.

14.4 logQ corrects sampling effects, not every popularity bias

When item j is sampled with probability $q(v_j)$, a sampled-softmax approximation may use

$$\widetilde{S}_{ij} = S_{ij} - \log q(v_j).$$

The precise correction must match the sampler: replacement, inclusion probabilities, duplicate handling, and positive treatment can matter. The name logQ does not justify inserting one formula into every contrastive objective.

The intuition is that popular items appear in sampled candidate sets more frequently, and that frequency affects what the training objective learns.

But a sampling correction does not automatically remove exposure bias, position bias, or a recommendation feedback loop. If nearly all observed positives are popular, logQ does not create evidence about unobserved tail preferences.

Future extensions might consider true unclicked exposures, position, watch quality, and propensity weighting. An unclicked exposure is still not certain dislike; a low-ranked click does not justify an arbitrary positive weight without a model of the observation process.

14.5 A training-record schema and API boundary

A useful training record may contain:

```
user_id / video_id / session_id / scenario
prediction_time / label_time
user_features_as_of_prediction_time
video_features_as_of_prediction_time
history_before_prediction_time
label / sample_weight / sampling_metadata
```

A batch ID is usually produced by the loader or sampler. It does not need to be a permanent predictive feature.

A PyTorch model can inherit from `nn.Module` and implement `forward`; the loss can be defined separately. Training uses autograd and an optimizer. Serving uses evaluation mode and `no_grad()` or `inference_mode()` where appropriate. **Calling `eval()` does not itself disable gradients.**

15. Appendix: significance, AUROC/NDCG, and feature importance

15.1 Before explaining an offline-online discrepancy

An observed offline improvement raises an earlier question: **Is the improvement reliable, and is its magnitude practically meaningful?** Only then should we explain why online outcomes do or do not follow.

This does not make p<0.05 the sole decision rule. Effect size, uncertainty, power, business cost, and a predefined decision protocol all matter.

15.2 p-values, alpha, and Z

For

$$H_0 : \mu_A = \mu_B,$$

construct

$$Z = \frac{\bar{X}_B - \bar{X}_A}{SE(\bar{X}_B - \bar{X}_A)}.$$

If the null distribution is approximately standard normal, a two-sided test at $\alpha = 0.05$ rejects when $|Z| > 1.96$. A prespecified one-sided test rejects in the chosen positive direction when Z>1.645.

The p-value is the probability, under the null, of an outcome at least as extreme as the observed statistic. It is not the probability that the null is true or the probability of rejecting the null.

Alpha is a preselected type-I error level. Comparing p with alpha and comparing the statistic with the corresponding critical quantile are two expressions of the same test rule.

15.3 CTR, NDCG, and AUROC have different aggregation structures

Metric	Unit-level contribution and aggregation
CTR	A mean of 0/1 outcomes when each independent unit has one opportunity; multiple exposures per user require ratio and clustering considerations
NDCG	Usually calculated per query and then averaged
AUROC	A ranking statistic over positive-negative pairs; a single record does not have its own independent AUC

For scores s,

$$\widehat{\mathrm{AUC}} = \frac{1}{n_+ n_-} \sum_{i:y_i=1} \sum_{j:y_j=0} \left[\mathbf{1}(s_i > s_j) + \tfrac{1}{2}\mathbf{1}(s_i = s_j) \right].$$

Pairs share observations, so the $n_+ n_-$ pairs are not independent experimental units.

For two fixed models evaluated on the same data, paired bootstrap samples the same evaluation units for both models in every replicate. Calculate both AUCs and their difference, then estimate uncertainty from the replicated differences. User-level dependence may call for user-cluster resampling.

DeLong's method can also estimate the variance of an AUC difference. Bootstrap is not mandatory merely because the metric is AUROC.

15.4 "A thousand genuinely independent test sets would work too"

Yes. Calculate a paired AUC difference for each independent set, then analyze those differences.

The target is then the average set-level AUC difference under that grouping protocol. Averaging subset AUCs is generally not the same as calculating one pooled AUC: the pooled calculation also includes positive-negative pairs across subsets. Set sizes, prevalence, and sources matter.

Resampling a fixed test set addresses finite-evaluation-sample uncertainty. It does not replace retraining with multiple seeds to assess training variability.

15.5 Permutation testing versus permutation importance

Permutation testing builds a null distribution by shuffling assignments or labels under a valid exchangeability or randomization scheme. It is a way to estimate a p-value, not a technique that creates extra information when the number of units is small.

A Z-test uses an approximate theoretical null distribution; a permutation test constructs one through admissible rearrangements. Large traffic usually reduces sampling variation, but does not make that variation irrelevant.

Permutation importance holds a model fixed, shuffles a feature, and measures the performance change. It asks how much the model relies on that feature.

For an embedding feature, shuffle the whole vector between records, keeping its internal coordinates together:

```
Original A: [other features A, embedding A, label A]
Shuffled A: [other features A, embedding B, label A]
```

Do not independently shuffle each dimension unless that is the specific intervention being studied. Repeated shuffles can reveal stability.

Correlation complicates interpretation. Shuffling a user embedding while retaining its user ID can generate unnatural combinations; interchangeable features can make individual importance look small. This is fixed-model reliance, not causal value and not the effect of removing a feature and retraining.

Tree models also expose split-count or gain importance; neural models can use gradient-based attribution. These methods do not all answer the same question.

16. Where Part 2 should resume

Model	Relationship represented	Main understanding reached
Two towers	User-item matching space	Independent encoding and serving contracts
Historical self-attention	Context and order within behavior history	A final state can encode the available past
FM	Pairwise category interactions	Factorized latent coefficients instead of a giant independent pair table
DeepFM	Explicit low-order and implicit complex interactions	Parallel FM and original-embedding MLP branches
Wide & Deep	Concrete combination memory and representation generalization	Chosen cross features with trainable coefficients
DCN v1	Bounded-degree explicit coordinate crosses	Expanding the input-dependent scale reveals products
DCNv2	More flexible coordinate-specific interactions	Matrix mixing and optional low-rank factorization
DLRM	Nonlinear relationships among pair scores	Retain each inner product for the top MLP
DIN	Candidate-specific historical interest	The candidate determines history relevance

The next part should finish the models before expanding the loss taxonomy:

1. **DIEN:** behavior is not identical to latent interest; how interest evolution and auxiliary supervision are modeled.
2. **SASRec alignment:** connect the already discussed causal history encoder to a named sequential recommender, focusing on its training target rather than relearning Transformers.
3. **Multi-interest retrieval / MIND:** generate multiple user vectors, assign history to them, train them, and retrieve with them.
4. **Losses:** pointwise, pairwise, and listwise objectives; BCE versus softmax CE; RankNet/BPR; margin and hinge; temperature; hard negatives; multiple positives; auxiliary objectives. Familiar topics need alignment rather than a complete restart.

The same reasoning discipline should guide each step:

> What is an example? What relationship should be learned? What is the most natural equation? What fails or becomes expensive? What structural or objective change addresses that? Do gradients and serving constraints still make sense?

References and reading order

The teaching order is not publication order. The original papers define the architectures; the reconstructed motivations here are a learning route, not a claim about the authors' exact discovery process.

1. Steffen Rendle. **Factorization Machines.** ICDM 2010. Official project and publication list; DOI.
2. Cheng et al. **Wide & Deep Learning for Recommender Systems.** 2016. Paper.
3. Guo et al. **DeepFM: A Factorization-Machine based Neural Network for CTR Prediction.** 2017. Paper.
4. Wang et al. **Deep & Cross Network for Ad Click Predictions.** 2017. Paper.
5. Wang et al. **DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems.** Preprint.
6. Naumov et al. **Deep Learning Recommendation Model for Personalization and Recommendation Systems.** 2019. Paper.
7. Zhou et al. **Deep Interest Network for Click-Through Rate Prediction.** KDD 2018; preprint first submitted in 2017. Paper.
8. Hu et al. **LoRA: Low-Rank Adaptation of Large Language Models.** Paper.
9. Janson et al. **Stabilizing Native Low-Rank LLM Pretraining.** ICML 2026. Paper.

For Part 2:

- Zhou et al. **Deep Interest Evolution Network for Click-Through Rate Prediction.** AAAI 2019. Paper.
- Kang and McAuley. **Self-Attentive Sequential Recommendation.** ICDM 2018. Paper.
- Li et al. **Multi-Interest Network with Dynamic Routing for Recommendation at Tmall.** CIKM 2019. Paper.

The central lesson of Part 1 is that a model is a structured way to represent relationships in data. Derive the relationship first, then read how a paper implements and validates it. The equations become much easier to understand that way.